National College of Ireland

Cloud Platform Programming

Introduction to Cloud Concepts.

Introduction to Developing on Cloud.

Outline

- Introduction to Cloud Concepts
- Working with a Cloud SDK
  - Errors and Exceptions
- Application and Infrastructure Monitoring
- Review
  - Systems Development Lifecycle
  - Versioning Control

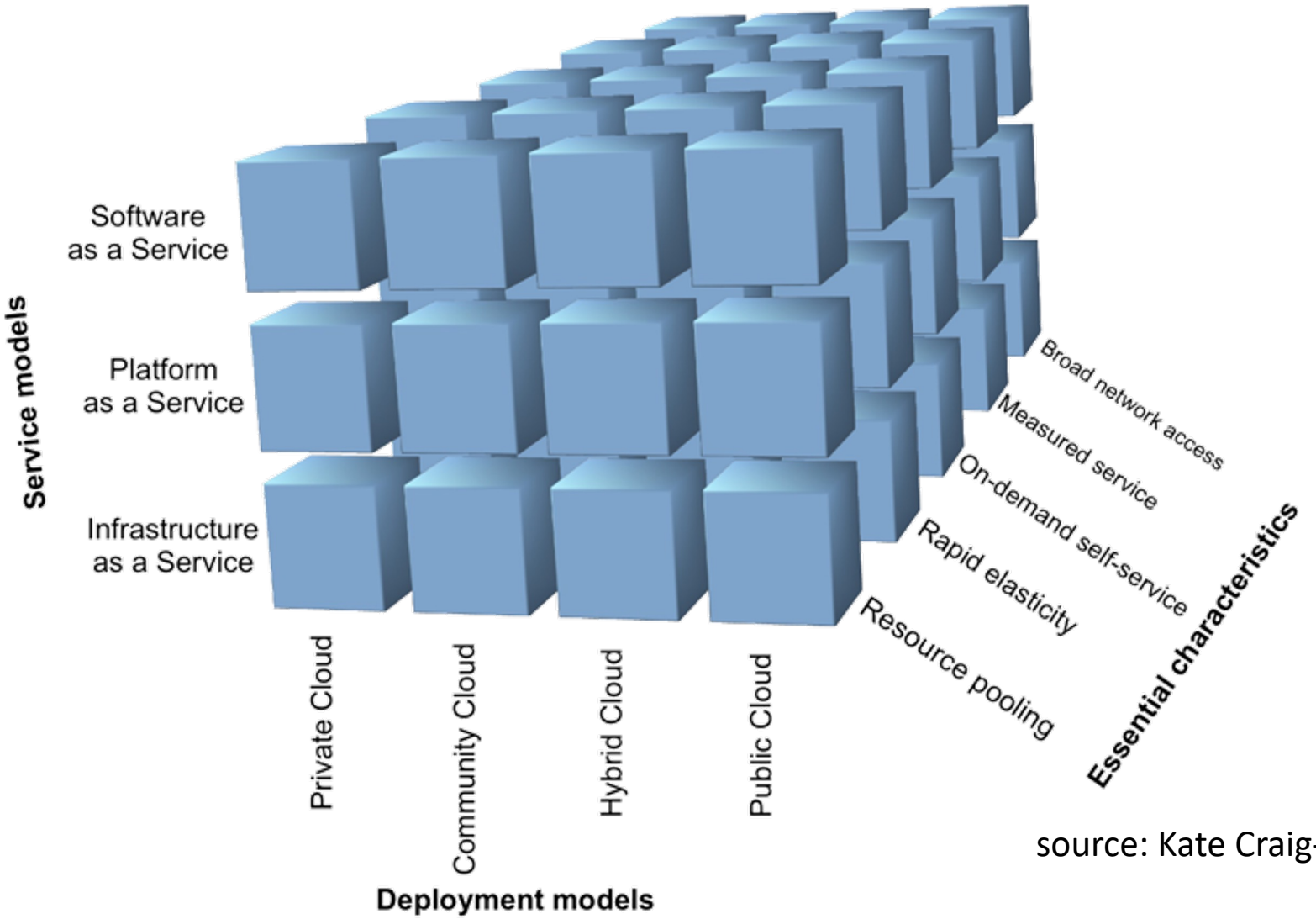
Cloud Computing

- NIST Definition:

  "Cloud computing is a model for enabling ***ubiquitous****,* ***convenient****,* ***on-demand network access to a shared pool of configurable computing resources*** *(e.g., networks, servers, storage, applications, and services) that can be* ***rapidly provisioned*** *and released with minimal management effort or service provider interaction."*

source: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf

Cloud Computing



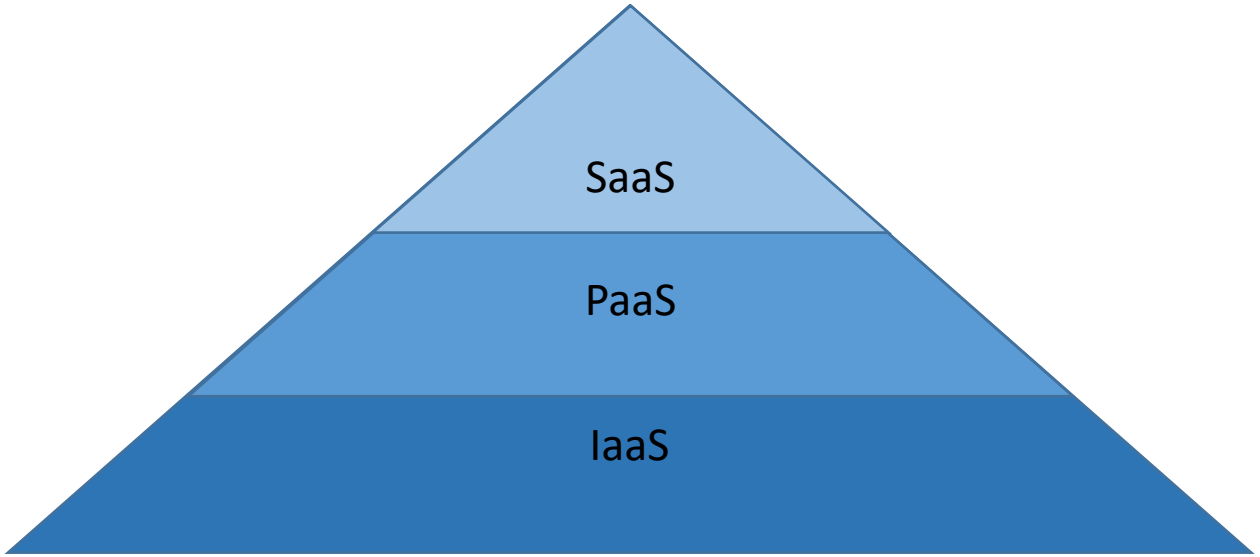
source: Kate Craig-Wood, NIST, and G-Cloud

Cloud Computing – Essential Characteristics

- On-demand self-service
- Resource pooling
- Rapid elasticity
- Measured Service
- Broad network access

Cloud Computing – Service Models

- Infrastructure as a Service
- Platform as a Service
- Software as a Service



Infrastructure as a Service

- Offers highly scalable and automated compute resources
- Resources: Data Storage / Networking/ Servers
  - Most cloud vendors are responsible for managing these resources
- The users or purchasers will be responsible for handling resources such as applications, data, runtime and middleware

Infrastructure as a Service

- Products & Services of IAAS:
- Pros
  - The Cloud Provides the Infrastructure
  - Enhanced Scalability – Dynamic Workloads are Supported
  - IAAS is Flexible
- Cons
  - Security Issues
  - Network & Service Delays
  - SLAs 99.98% Availability







Platform as a Service

- A platform for developers, made up of programming language execution environment, an operating system, a web server & database.
- Encapsulates the environment where users can build, compile & run their programs without worrying of the underlying infrastructure
- In this Cloud Computing Model, you manage data & the application resources; all other resources are managed by the cloud vendor

Platform as a Service

- Dev Environments
- Pros
  - Cost Effective Rapid Development (Scalability)
  - Faster Market For Developers
  - Easy Development of Web Applications
  - Private & Public Deployment is Possible
- Cons
  - Developers Are Limited to Providers Languages & Tools
  - Migration Issues: Vendor Lock In







AWS Elastic Beanstalk

Software as a Service

- Use the provider’s application running on a cloud infrastructure
- On Demand
- Pay-Per-Use
- Don't need to install software on end devices
- Makes cloud computing a lot cheaper than traditional methods
- All computing resources for SAAS required to run the software are managed by the provider
- Accessible via Web Browser or Lightweight Client Applications

Software as a Service

- CRM / ERP / Google Eco Sys
- Pros
  - Accessible from any platform
  - Work From Anywhere
  - Collaborative Working
  - Vendor Provided
  - Multi tenancy
- Cons
  - Portability & Browser Issues
  - Internet Performance
  - Compliance Restrictions







Cloud Computing – Deployment Models

- Private Cloud
- Community Cloud
- Public Cloud
- Hybrid Cloud

Cloud Computing – Deployment Models

- Private Cloud
  - The cloud infrastructure is provisioned solely for the use by an organization
  - Managed by the organization or a third party
  - May be located on premise or off premise
- Community Cloud
- Public Cloud
- Hybrid Cloud

Cloud Computing – Deployment Models

- Private Cloud
- Community Cloud
  - The cloud infrastructure is shared by a specific community of consumers from organizations that share similar concerns (e.g. mission, security requirements, policy, and compliance considerations)
  - Managed by the organizations or a third party
  - May be located on premises or off premises
- Public Cloud
- Hybrid Cloud

Cloud Computing – Deployment Models

- Private Cloud
- Community Cloud
- Public Cloud
  - The cloud infrastructure is provisioned for the use by general public or a large industry group
  - Is located on the premises of the cloud provider
- Hybrid Cloud

Cloud Computing – Deployment Models

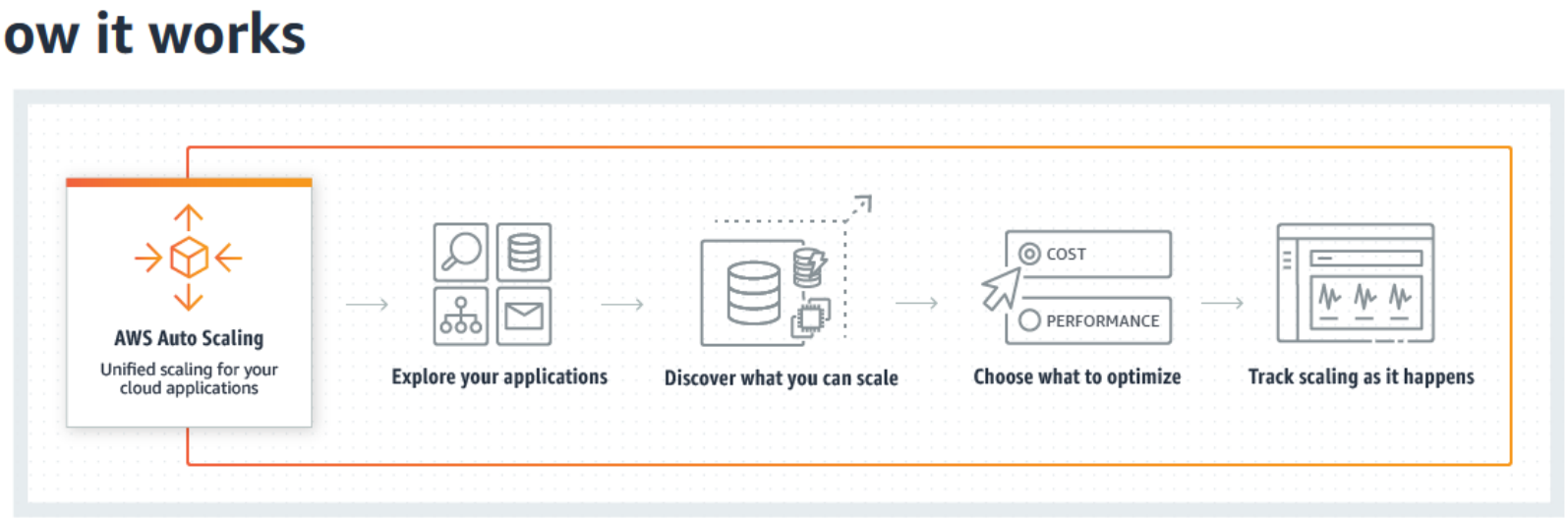
- Private Cloud
- Community Cloud
- Public Cloud
- Hybrid Cloud
  - The cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public)

Cloud Core Services

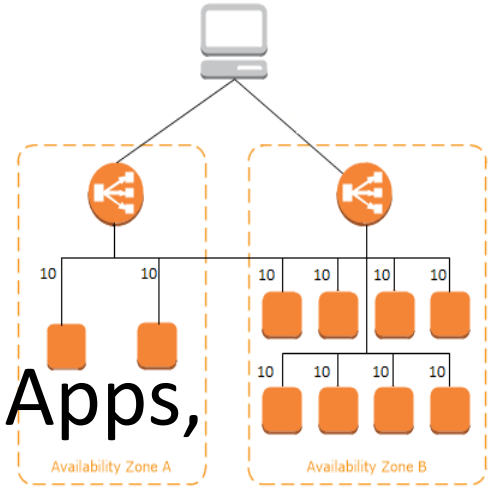
- Primarily we consider cloud as a provider of resources
  - Compute – e.g., EC2
  - Storage – e.g., S3
  - Databases – e.g., SQL and NoSQL
  - Network – VPN / Secure Tunnel / VPC

Cloud Services – Scaling/Elasticity

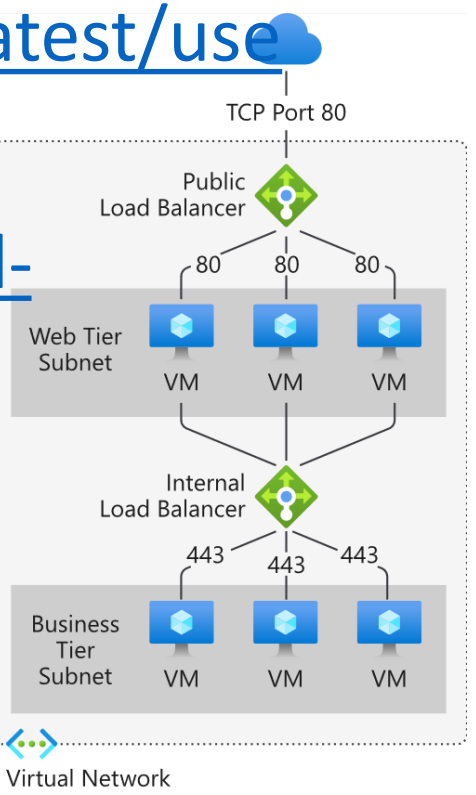
- Auto Scaling (Expanding and Contracting Based on App needs)
- e.g., AWS Auto Scaling https://aws.amazon.com/autoscaling/



Cloud Services – ELB



- Elastic Load Balancing – Redirecting Traffic to WebApps, servers, etc.
  - e.g., AWS ELB https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/how-elastic-load-balancing-works.html
  - Google Load Balancing https://cloud.google.com/load-balancing
  - Azure Load Balancer https://docs.microsoft.com/en-us/azure/load-balancer/load-balancer-overview



National College of Ireland

Introduction to Developing on Cloud

Working with a Cloud SDK

- Typically, the cloud service providers (CSP) offer software development kits (SDKs) for their services in a variety of programming languages and technologies
- We use the SDKs to call/invoke cloud services from our own applications

Working with a Cloud SDK

Examples of AWS SDKs

- SDKs
  - Python, Java, Ruby, .Net, C++, Node.js, etc.
- Mobile SDKs
  - Android, iOS, Mobile Web, Unity, Xamarin, etc.
- IoT Device SDKs
  - Embedded C, JavaScript, Arduino Yun, Java, Python, C++

source: https://aws.amazon.com/getting-started/tools-sdks/

Working with a Cloud SDK

AWS SDKs provide two levels of APIs for the AWS services

- Client (or "low-level") APIs
- Resource APIs

source: https://aws.amazon.com/sdk-for-python/

Working with a Cloud SDK

AWS SDKs provide two levels of APIs for the AWS services

- Client (or "low-level") APIs
  - Each client class has a direct mapping to the corresponding AWS service
  - Provide one-to-one mappings to the underlying HTTP API operations
    - Has one method per service operation
  - Have objects for request parameters and response data

source: https://aws.amazon.com/sdk-for-python/

Working with a Cloud SDK

AWS SDKs provide two levels of APIs for the AWS services

- Resource APIs
  - Has one class per conceptual resource
  - Provide resource objects and collections to access attributes and perform actions
  - Defines service resources and individual resources
    - e.g. service resources: S3, SQS
    - e.g. individual resources bucket, queue

source: https://aws.amazon.com/sdk-for-python/

Errors and exceptions

- When an API call is made to an AWS service the following takes place
  - A request is sent to the service
  - The service executes the request, and
  - Then the service returns a response
    - The response includes also an HTTP response code
- The HTTP response code shows if the operation is successful or there is an error

Errors and exceptions

- The HTTP response code shows if the operation is successful or there is an error
- There are two types of API errors
  - Client errors
  - Server errors

Errors and exceptions

- There are two types of API errors
  - **Client errors** - indicated by a 4xx HTTP response code
    - Client errors show that there is a problem with the client request
      - e.g., bucket does not exist, authentication failure, missing required parameters
    - We have to handle the error in our application
  - **Server errors** - indicated by a 5xx HTTP response code
    - Need to be resolved by Amazon
    - We can resubmit/retry the request until it succeeds

Errors and exceptions

- AWS SDK for Python (Boto 3)
  - The AWS SDK for Python throws the botocore.exceptions.ClientError.

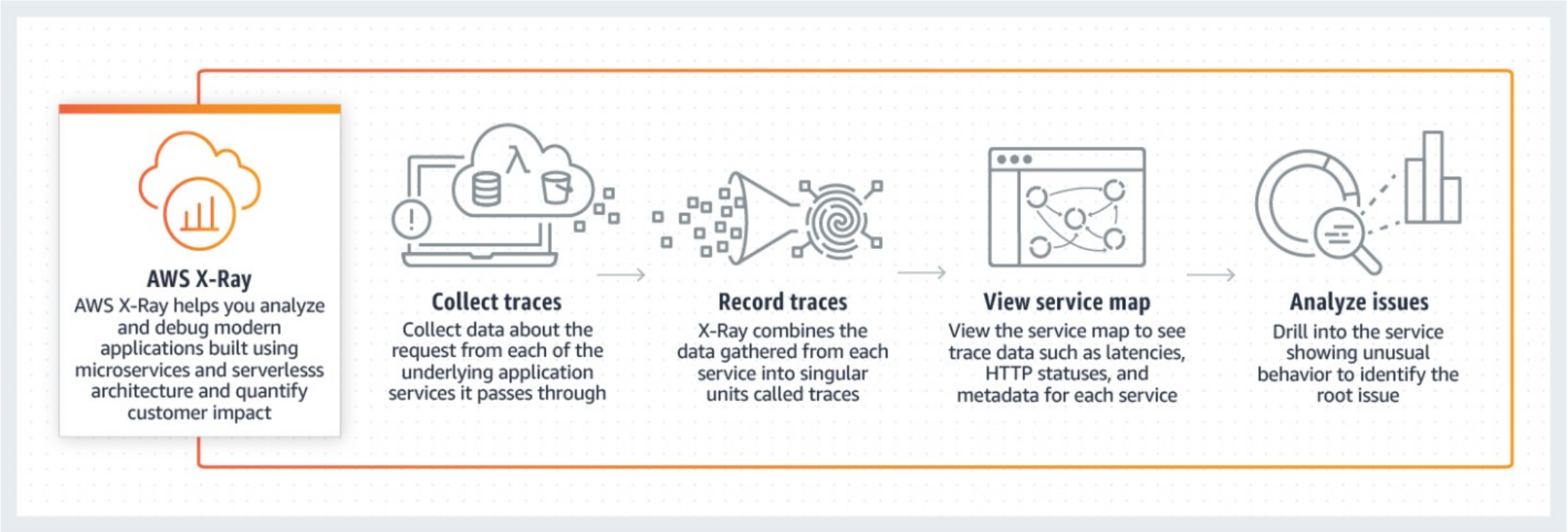
source: https://boto3.amazonaws.com/v1/documentation/api/latest/guide/error-handling.html

Application and Infrastructure Monitoring

- AWS X-Ray
- Amazon CloudWatch
- AWS CloudTrail

AWS X-Ray

- A service that enables to identify and troubleshoot the root cause of performance issues and errors



source: https://aws.amazon.com/xray/

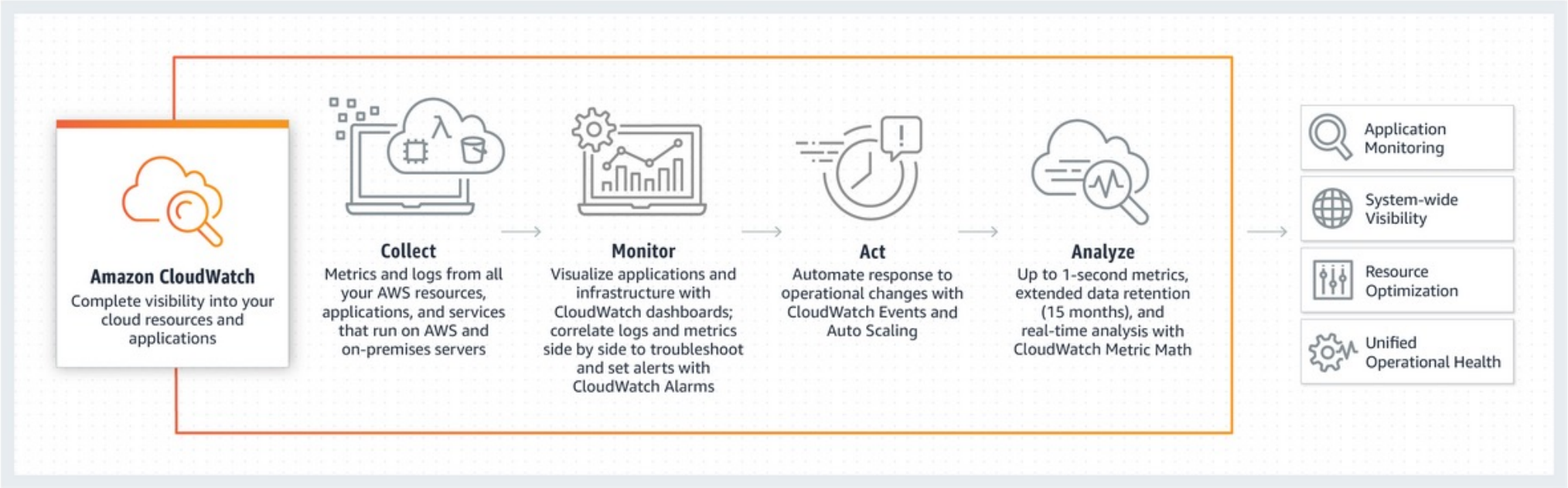
AWS X-Ray

- Identify errors
- Identify performance issues
- Enable end-to-end view of requests as they travel through your application
- Shows a map of your application's underlying components

source: https://aws.amazon.com/xray/

Amazon CloudWatch

- Amazon CloudWatch is a monitoring and observability service



source: https://aws.amazon.com/cloudwatch/

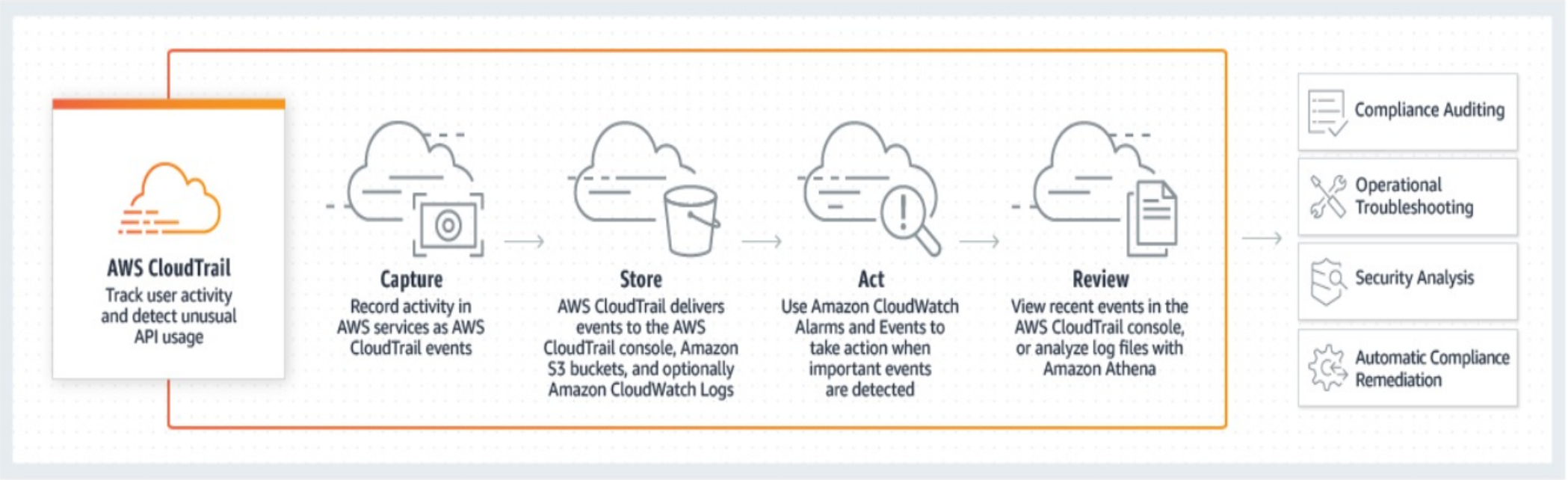
Amazon CloudWatch

- Monitors your applications and stores logs
- Collects and tracks metrics
- Enables us to set alarms and react to changes
  - Respond to system-wide performance changes
- Provide a unified view of operational health
  - e.g., graphs, statistics

source: https://aws.amazon.com/cloudwatch/

AWS CloudTrail

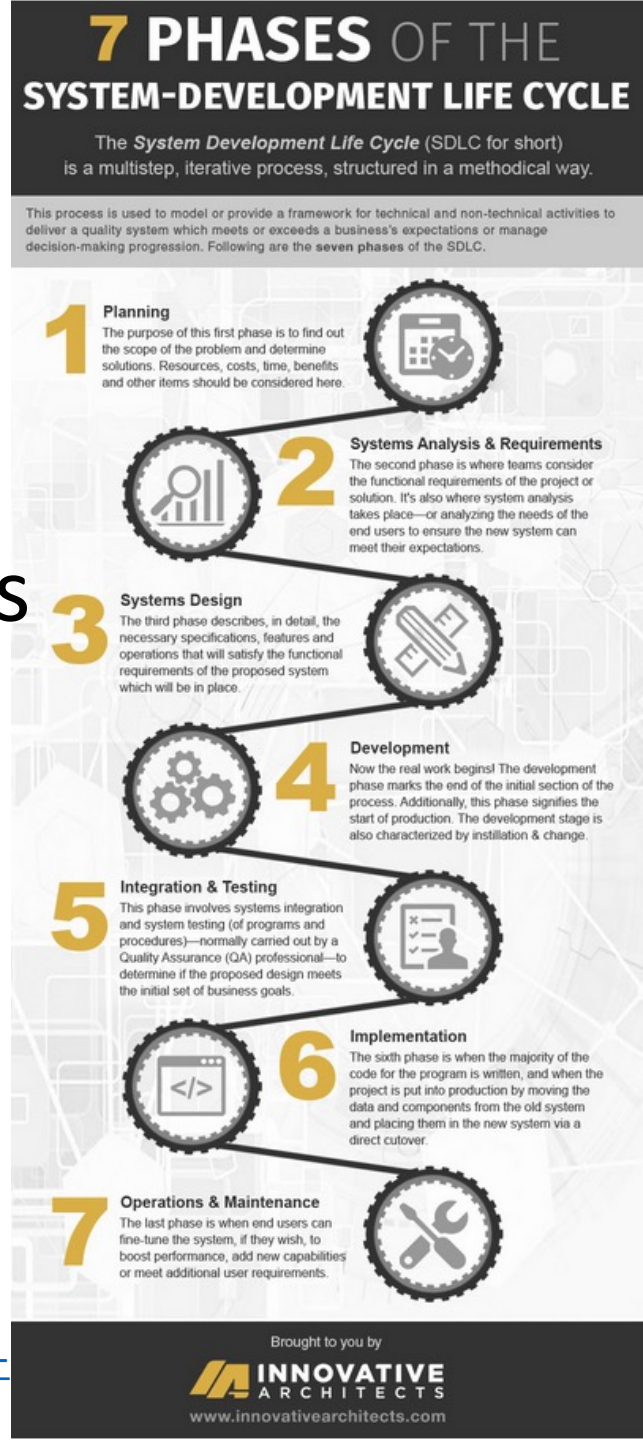
- AWS CloudTrail is a service that enables governance, compliance, operational auditing, and risk auditing of your AWS account



source: https://aws.amazon.com/cloudtrail/

The Software Development Lifecycle (SDLC)

- Plan
- Define – Systems Analysis & Requirements
- Systems Design
- Development
- Integration & Testing
- Deployment
- Operations & Maintenance

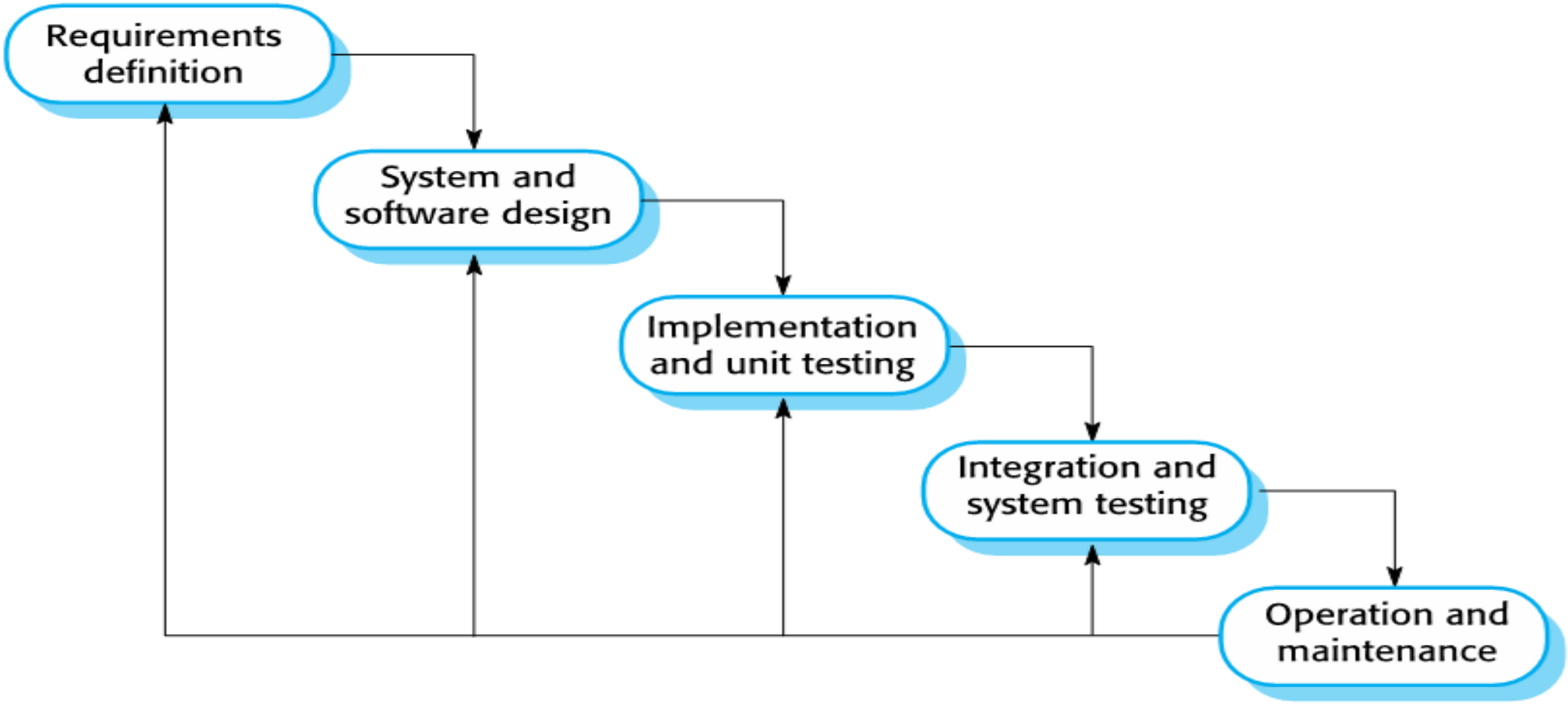


source: https://www.innovativearchitects.com/KnowledgeCenter/basic-IT-systems/system-development-life-cycle.aspx

SDLC Methodologies

- Waterfall (or traditional) methodology
  - The waterfall model describes a sequential development method in which each development phase has distinct goals and tasks that must completed before the next phase can begin
- Agile software development methodology
  - Software is developed iteratively in short time periods called sprints

SDLC – Waterfall Model



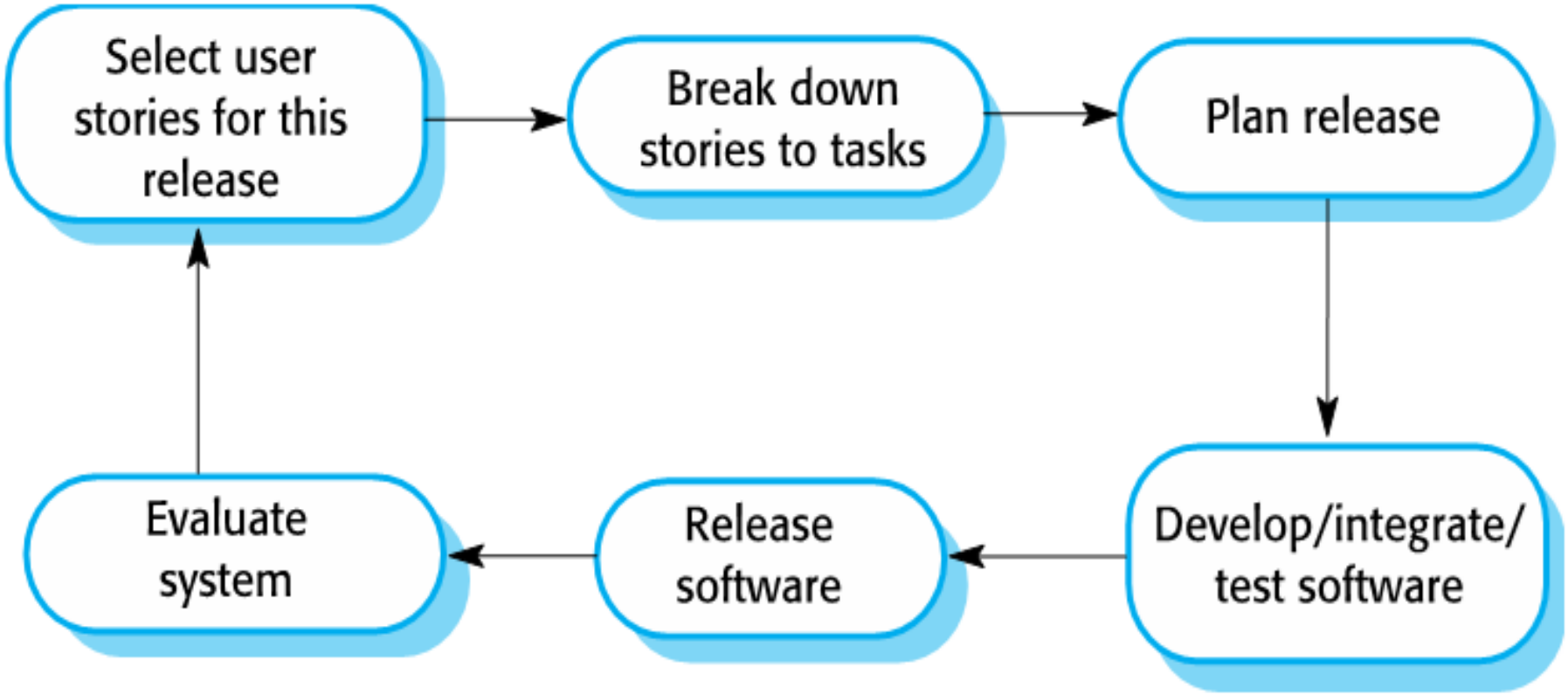
source: Ian Sommerville. Software Engineering 10th Edition, chapter 2.

SDLC – Waterfall Model

- Disadvantages
  - Software artefacts produced later in the development lifecycle
  - It is difficult to cope with changes, as typically, a phase should finish before moving to the next phase

Source: Software Engineering 10th Edition. Ian Sommerville.

SDLC – Extreme Programming Model – An Example of Agile Methodology



source: Ian Sommerville. Software Engineering 10th Edition, chapter 3.

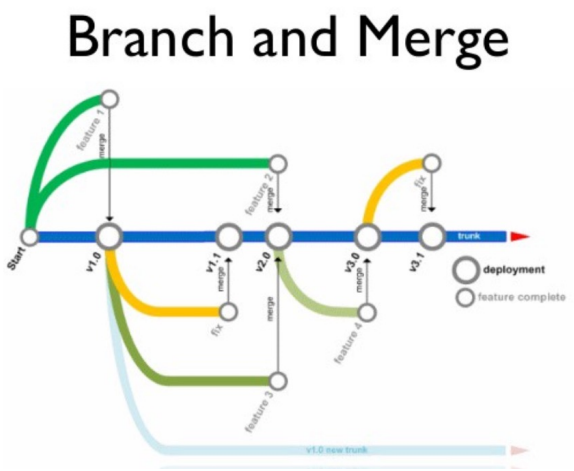
SDLC – Agile Model

- Advantages
  - Incremental development is enabled by small, frequent system releases
  - Change is supported via regular system releases

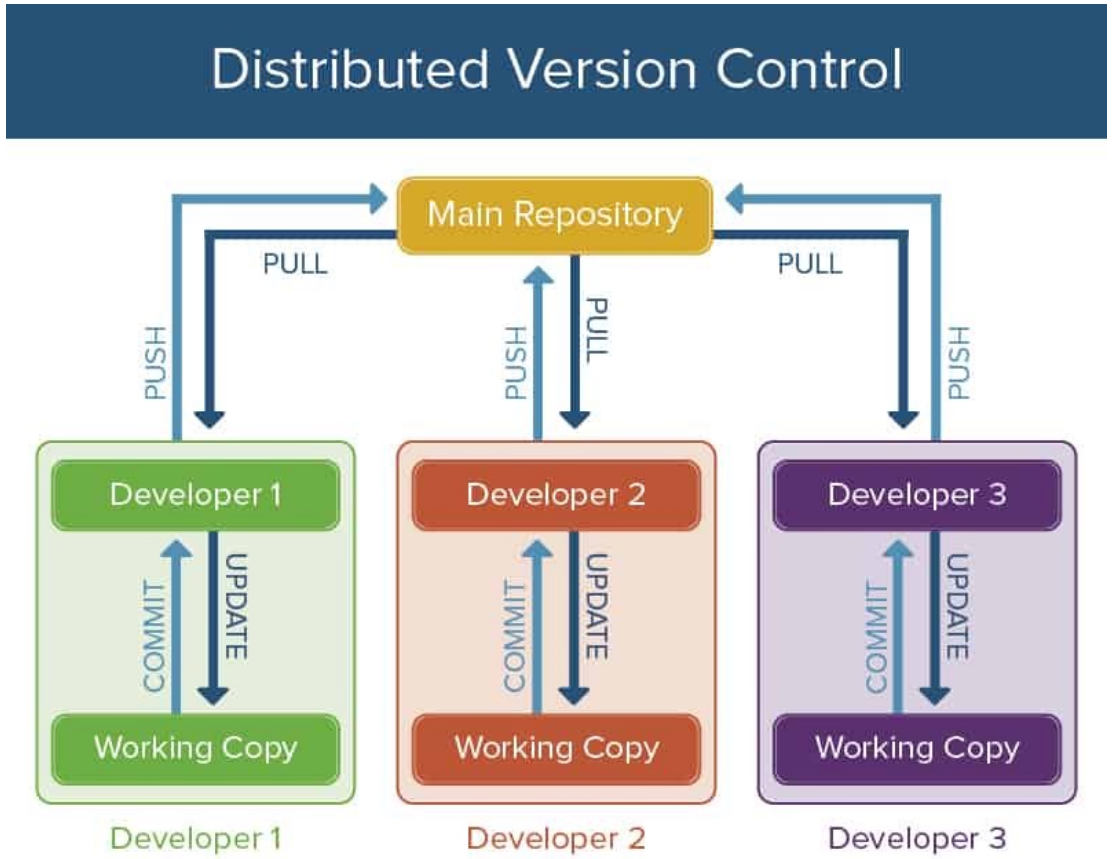
Versioning Control

- What is it ?
- Services Available

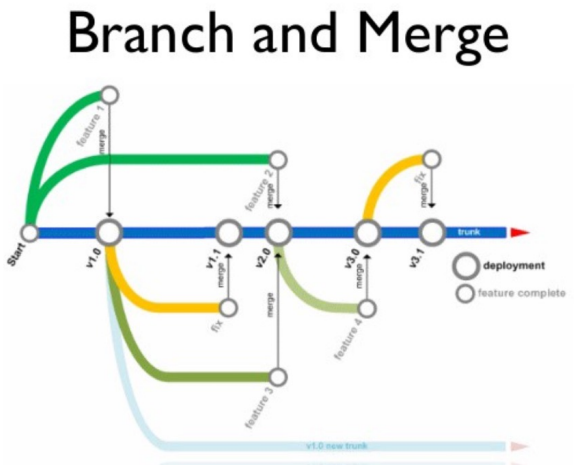
The importance of version control systems for tracking changes cannot be overstated, and companies should choose the one that best suits their needs.



Versioning Control



The importance of version control systems for tracking changes cannot be overstated, and companies should choose the one that best suits their needs.



Versioning Control

- If you are planning to work on a big software development project that requires collaboration of different team members and need frequent changes, then you need to use a version control system

- A version control system allows users to keep track of the changes in software development projects, and enable them to collaborate on those projects

- Using it, the developers can work together on code and separate their tasks through branches

Versioning Control

- The main advantages of using a version control system include streamlining the development process, management of code for multiple projects and keeping a history of all changes within a code

- A version control software saves all the changes in a repository. Hence, if the developers make a mistake, they can undo it. At the same time, they can compare the new code with a previous version(s). This can reduce human errors and unintended consequences

- Additionally, it can be integrated with several software development tools like PaaS providers, integrated development environments (IDE) and build automation tools

Versioning Control

- Services Available
  - GitHub
  - BitBucket
  - GitLab
  - Apache Subversion
  - AWS CodeCommit

Resources

- The NIST Definition of Cloud Computing: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf
- The Seven Phases of the System-Development Life Cycle https://www.innovativearchitects.com/KnowledgeCenter/basic-IT-systems/system-development-life-cycle.aspx
- AWS SDK for Python (Boto3) https://aws.amazon.com/sdk-for-python/
- AWS X-Ray https://aws.amazon.com/xray/
- Amazon CloudWatch https://aws.amazon.com/cloudwatch/
- AWS CloudTrail https://aws.amazon.com/cloudtrail/

Acknowledgements

- Some of material presented was adapted from lecture notes provided courtesy of Sean Heeney